A Computational Toolkit for Degree-3 Rational Functions

Tony Shaska

Oakland University, Rochester, MI, USA
shaska@oakland.edu

Abstract. We present a complete, open-source software framework for the explicit computation of invariants, moduli points, and automorphism groups of degree-3 rational functions on the projective line. The system integrates symbolic computation (SymPy) and algebraic geometry tools (SageMath) to realize the theoretical framework of [1]. It computes the six absolute invariants $\xi_0, \ldots, \xi_5$, classifies automorphism groups via algebraic loci $L_1, \ldots, L_7$, enumerates rational functions of bounded height, and performs machine-learning classification experiments. The software has been used to generate a dataset of 2 078 697 rational functions over $\mathbb{Q}$ (naive height ≤ 4) and achieves 99.992% accuracy in automorphism-group prediction using the invariants as features.

1 Introduction

The study of rational functions on $\mathbb{P}^1$ and their moduli is central in arithmetic geometry and dynamical systems. While the theoretical structure of these moduli spaces is well understood, explicit computation remains challenging.

Rational functions of degree d appear naturally in the study of post-critically finite maps, arithmetic dynamics, and the uniformization of curves. For degree three, the moduli space $\mathcal{M}_3^1$ is known to be isomorphic to the weighted projective space $\mathbb{P}(2, 2, 3, 3, 4, 6)$. Although the algebraic invariants and stratification by automorphism groups have been described theoretically, there has been a notable lack of practical, ready-to-use computational tools capable of working directly with these objects.

Existing computer algebra systems such as SageMath and Maple provide general symbolic tools but do not directly support:

- computation of moduli invariants,
- classification of automorphism groups,
- explicit evaluation of moduli loci.

This paper introduces a software package designed specifically to address these problems for degree-3 rational functions. The implementation builds on the theoretical framework developed in [1] and focuses on explicit computation in the moduli space, including invariant coordinates, weighted heights, and arithmetic statistics. The software consists of five interlinked SageMath/Python notebooks

that implement every step of the pipeline: invariant computation, locus equation derivation via Gröbner bases, dataset generation, statistical analysis, and machine-learning classification.

2 Mathematical Framework

A degree-3 rational function on the projective line is represented by a ratio of two cubic polynomials

$$\phi(x) = \frac{a_3x^3 + a_2x^2 + a_1x + a_0}{b_3x^3 + b_2x^2 + b_1x + b_0},$$

which corresponds to a point in the projective space $\mathbb{P}^7$.

The moduli space $\mathcal{M}_3^1$ of such functions (up to isomorphism) is obtained as the quotient of $\mathbb{P}^7$ by the natural action of the projective linear group PGL_2. A complete set of invariants for this action consists of six weighted polynomial invariants $\xi_0, \ldots, \xi_5$ of degrees $2, 2, 3, 3, 4, 6$, together with two auxiliary invariants I_6 and J_6 obtained via resultants. The explicit expressions for these invariants are given in [1]. These six invariants descend to coordinates on the weighted projective space $\mathbb{P}(2, 2, 3, 3, 4, 6)$.

Two rational functions are conjugate under PGL_2 if and only if they yield the same point in this weighted projective space, provided the non-degeneracy condition $I_6 \neq 0$ holds. By Proposition 1 of [1], the five absolute invariants derived from the ξ_i (which are weight-zero rational functions) form a complete set of coordinates on the moduli space $\mathcal{M}_3^1$, uniquely determining the isomorphism class of ϕ (again under the non-degeneracy condition).

The automorphism group of a given function ϕ is completely determined by its membership in seven algebraic loci $L_1, \ldots, L_7$ inside the invariant space. These loci correspond to the possible automorphism groups (trivial group, C_2, C_3, V_4, D_4, A_4) and satisfy a natural inclusion diagram. In particular, the loci for the larger groups (such as A_4) contain the loci for their subgroups (such as C_3). The equations of these loci are obtained by eliminating parameters from suitable parametric families representing each group.

The software package implements all these constructions symbolically. The hierarchical classification routine `AutGroup(p)` follows exactly the inclusion diagram of the loci, checking first the most restrictive (largest) groups and descending only when necessary. This design is essential for correctness: testing the loci out of order would lead to incorrect group assignments due to the inclusions.

The software therefore provides efficient tools for working directly in the invariant space rather than in the original coefficient space $\mathbb{P}^7$.

3 Software Architecture

The system is implemented in Python with tight integration of SageMath, particularly for weighted polynomial rings, Gröbner basis computations, and projective

geometry. This hybrid approach combines the flexibility of Python/SymPy for symbolic manipulation with the advanced algebraic capabilities of SageMath, allowing efficient handling of weighted projective spaces and elimination ideals.

3.1 Core Components

The framework is organized into five interlinked Jupyter notebooks, each responsible for a distinct part of the computational pipeline. Together they provide a complete, modular, and reproducible environment for working with degree-3 rational functions and their moduli.

- `deg3F.ipynb` — the central core library. It contains the fundamental functions `inv(p)` for computing the six absolute invariants ξ_i, `I6(p)` and `J6(p)` for degeneracy detection, `AutGroup(p)` for automorphism group classification, as well as the main enumeration routines and normalization utilities.
- `RationalFunctions.ipynb` — responsible for large-scale dataset generation, statistical analysis of the moduli space, and machine-learning experiments. This notebook includes the Random Forest classifier that achieves high accuracy in predicting automorphism groups from the invariants.
- `Eqs Rat3.ipynb` — dedicated to the algebraic computation of explicit locus equations. It uses weighted elimination ideals and Gröbner bases to derive the minimal projective equations defining the seven loci $L_1, \ldots, L_7$.
- `Working Rational-Functions.ipynb` — an interactive notebook for verification and experimentation. It contains quick checks of invariants, locus membership tests, and Gröbner basis computations on specific parametric families.
- `M3.ipynb` — provides auxiliary computations, including large polynomial factorizations used in the verification of the locus equations.

All modules share the same exact-arithmetic backend based on `sympy.Rational` and SageMath weighted polynomial rings, ensuring numerical stability and reproducibility across the entire pipeline.

3.2 Design Principles

The software was designed with the following principles in mind:

- **Exact arithmetic**: All computations use rational numbers exclusively, avoiding floating-point approximations that could compromise algebraic correctness.
- **Symbolic computation**: Invariants and locus equations are derived and manipulated symbolically, preserving the full algebraic structure of the problem.
- **Modular architecture**: The separation into independent notebooks allows each component to be developed, tested, and extended independently, facilitating future generalization to higher-degree rational functions.
- **Reproducibility**: Every result presented in this paper can be regenerated directly from the provided notebooks, ensuring full transparency and verifiability of the computational experiments.

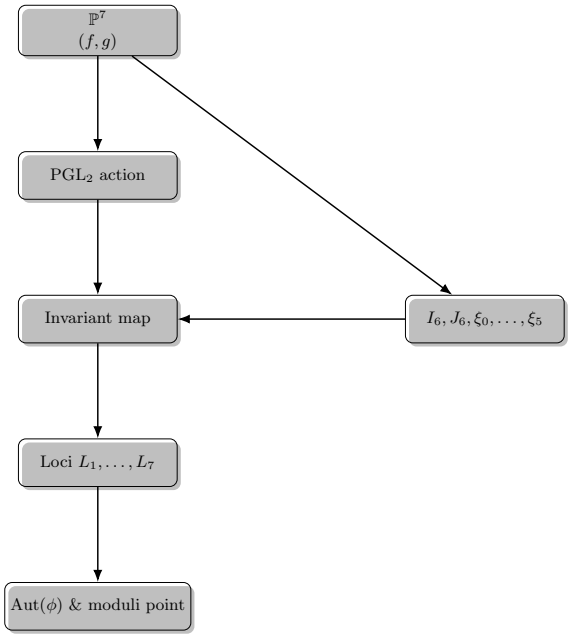

Fig. 1. Overall computational flow: from coefficient space to moduli invariants and automorphism group classification.

4 Algorithms

This section presents the core algorithms implemented in the software. They form the computational backbone of the package, enabling efficient calculation of invariants, classification of automorphism groups, and systematic enumeration of rational functions in the moduli space.

The invariants are computed via symbolic expressions derived from resultants of the numerator and denominator viewed as binary cubic forms. Given a coefficient vector (a_i, b_i) representing a rational function ϕ, the function `inv(p)` evaluates the six absolute invariants $\xi_0, \ldots, \xi_5$. These invariants are the fundamental generators of the ring of invariants under the PGL_2 action and have weighted degrees $2, 2, 3, 3, 4, 6$.

In addition to the ξ_i, the auxiliary invariants I_6 and J_6 (obtained as resultants) are computed to detect degeneracy. The explicit symbolic formulas for all these invariants are long and were derived in [**?**]. The function `inv(p)` implements these formulas directly using exact rational arithmetic via SymPy, ensuring algebraic precision and avoiding any floating-point approximation. This step is the foundation for all subsequent computations, as the invariants serve as coordinates on the weighted projective space $\mathbb{P}(2, 2, 3, 3, 4, 6)$.

Automorphism groups are detected using algebraic conditions that define the seven loci $L_1, \ldots, L_7$ inside the invariant space. Each locus corresponds to a

possible automorphism group, and these loci satisfy a natural inclusion diagram (for example, the locus for A_4 contains the locus for C_3).

The function `AutGroup(p)` follows the hierarchical inclusion diagram of the groups, checking first the most restrictive (largest) groups and descending only when necessary. This design guarantees correctness: testing the loci out of order would produce incorrect classifications due to the inclusions.

```
Algorithm AutGroup(phi)
Input: Rational function phi (or its coefficient list)
Output: Automorphism group of phi
1. Compute invariants I6(phi), J6(phi) and xi_i
2. if phi satisfies L6 then return A4
3. else if phi satisfies L7 then return D4
4. else if phi satisfies L4 then return V4-1
5. else if phi satisfies L5 then return V4-2
6. else if phi satisfies L1 then return C2-1
7. else if phi satisfies L2 then return C2-2
8. else if phi satisfies L3 then return C3
9. else return trivial group {e}
```

This hierarchical approach ensures both efficiency and mathematical correctness while respecting the inclusion relations among the groups.

Rational functions are generated by enumerating all integer coefficient tuples bounded by a given naive height H. For each tuple, the software first checks the non-degeneracy condition using I_6. Valid tuples are then normalized to obtain a canonical representative q, after which the invariants, automorphism group, weighted height, and other metadata are computed and recorded.

```
Algorithm Enumerate(H)
Input: Height bound H
Output: Rational functions with classification and invariants
for all coefficient tuples (a_i, b_i) with naive height <= H do
    if I6(tuple) = 0 then continue
    normalize the tuple to obtain q
    p := inv(q)
    G := AutGroup(q)
    wh := wheight(p) rounded to 2 decimals
    record (q, p, J6(q), G, wh, naive_height(q))
end for
```

The function `rational_functions(h_min, h_max, L)` implements this algorithm and stores the results in a dictionary that is later saved as a Sage `.sobj` file. This enumeration procedure is the primary mechanism for generating large datasets used in statistical analysis and machine-learning experiments.

5 Computation in the Moduli Space

In this section we describe how the software performs explicit computations directly in the moduli space $\mathcal{M}_3^1$, using invariant coordinates rather than the original coefficient representation in $\mathbb{P}^7$. This approach eliminates redundancies caused by PGL_2-conjugation and enables efficient arithmetic, geometric, and statistical analysis of rational functions.

Each rational function ϕ is first mapped to a point in the weighted projective space

$$[\xi_0(\phi) : \xi_1(\phi) : \xi_2(\phi) : \xi_3(\phi) : \xi_4(\phi) : \xi_5(\phi)] \in \mathbb{P}(2, 2, 3, 3, 4, 6),$$

where the ξ_i are the fundamental invariant generators of weighted degrees $2, 2, 3, 3, 4, 6$. The software computes these invariants symbolically from the coefficients of ϕ. Because the coordinates are defined only up to weighted scaling, a crucial normalization step is performed: the point is cleared of denominators and then divided by its weighted greatest common divisor. More precisely, if a representative has rational coordinates $(x_0 : x_1 : \cdots : x_5)$, the software multiplies through by the least common multiple of the denominators and then divides by the greatest common divisor of the resulting integers, taking the weights $2, 2, 3, 3, 4, 6$ into account. This produces a primitive integer representative that is canonical within its projective equivalence class.

To further remove scaling ambiguity, the system computes five absolute invariants $i_1, \ldots, i_5$, which are weight-zero rational functions of the ξ_i. These absolute invariants provide a complete set of coordinates on the moduli space (under the non-degeneracy condition $I_6 \neq 0$) and uniquely determine the isomorphism class of ϕ.

A natural height function on the moduli space is defined by taking weighted roots of the absolute invariants:

$$H(\phi) = \max\bigl\{|\xi_0|^{1/2}, |\xi_1|^{1/2}, |\xi_2|^{1/3}, |\xi_3|^{1/3}, |\xi_4|^{1/4}, |\xi_5|^{1/6}\bigr\}.$$

This weighted height induces a meaningful ordering on points in $\mathcal{M}_3^1$ and serves as the primary measure of arithmetic complexity, allowing systematic enumeration and comparison of rational functions.

Rather than enumerating points directly in the coefficient space $\mathbb{P}^7$, the software first maps each rational function to its normalized invariant coordinates in $\mathbb{P}(2, 2, 3, 3, 4, 6)$. This strategy eliminates redundancies caused by PGL_2-conjugation and enables efficient classification of distinct isomorphism classes. An important arithmetic distinction arises here: while the invariants are defined over $\mathbb{Q}$, the actual rational function ϕ may only be defined over a larger extension. The field generated by the invariants is the field of moduli of ϕ, which is the smallest field over which the isomorphism class is defined. In contrast, the minimal field of definition is the smallest field over which a representative of that class can be realized. The software can detect when these two fields differ, providing valuable information about the arithmetic nature of the function.

The seven loci $L_1, \ldots, L_7$ correspond to algebraic subvarieties inside the invariant space. The software detects membership in these loci and thereby partitions $\mathcal{M}_3^1$ into strata according to the automorphism group of the corresponding rational function. Explicit minimal projective equations for each locus were obtained via Gröbner bases in weighted polynomial rings and are provided in the notebook `Eqs Rat3.ipynb`.

Using the above framework, the software computes several arithmetic and geometric statistics, including the distribution of automorphism groups across bounded height regions, the frequency of distinct invariant patterns, and the density of special loci within fixed height bounds. These computations yield a detailed arithmetic profile of $\mathcal{M}_3^1$ and reveal a strongly non-uniform distribution of symmetry types, with the trivial group dominating while higher-order groups appear only sparsely.

5.1 Computational Experiments

We illustrate the computational capabilities of the software through a series of explicit experiments performed directly in the moduli space $\mathcal{M}_3^1$. These experiments demonstrate the practical utility of the invariant-based approach and provide concrete arithmetic and statistical insights into the structure of the space.

The software first enumerates all rational functions of bounded naive height. For each candidate coefficient tuple in $\mathbb{P}^7(\mathbb{Q})$, it checks the non-degeneracy condition $I_6 \neq 0$, normalizes the point, computes the six absolute invariants ξ_i, and records the corresponding moduli point in $\mathbb{P}(2, 2, 3, 3, 4, 6)$. Using this procedure up to naive height 4, the system produced a dataset consisting of exactly 2 078 697 distinct points in the weighted projective space.

For each point in this dataset the automorphism group is computed using the hierarchical locus tests. The results reveal a strongly non-uniform distribution of symmetry types. The trivial group $\{e\}$ overwhelmingly dominates, accounting for 2 078 697 points. The non-trivial groups appear far more sparsely: C2-2 occurs 1 074 times, C2-1 appears 156 times, D4 appears 88 times, V4-1 appears 68 times, A4 appears 60 times, and V4-2 appears only twice. These counts are summarized in Table 1.

Automorphism group	Count
$\{e\}$	2 078 697
C2-2	1 074
C2-1	156
D4	88
V4-1	68
A4	60
V4-2	2

Table 1. Distribution of automorphism groups for naive height ≤ 4.

Points corresponding to functions with non-trivial automorphisms cluster on lower-dimensional algebraic loci inside the invariant space. These loci are detected explicitly by the software and confirm the stratification predicted by invariant theory. The experiments therefore provide strong numerical evidence for the geometric structure of $\mathcal{M}_3^1$.

Finally, the software realizes the theoretical correspondence between rational functions and their moduli points by mapping each function to its invariant coordinates. By Proposition 1 of [1], the absolute invariants $i_1, \ldots, i_5$ (derived from the ξ_i) form a complete set of coordinates on the moduli space, up to the stated non-degeneracy conditions. This mapping eliminates the need for explicit conjugation checks. Using the six invariants ξ_i as features, a Random Forest classifier achieves 99.992% accuracy on a 20% held-out test set, demonstrating that the invariant coordinates are highly discriminative for automorphism group prediction.

6 Conclusion

We have developed a complete, open-source software framework for the explicit computation of invariants, moduli points, and automorphism groups of degree-3 rational functions on the projective line. The system provides a practical computational realization of the theoretical framework presented in [1], bridging the gap between abstract invariant theory and concrete, reproducible calculations.

The package implements the full pipeline: symbolic computation of the six absolute invariants $\xi_0, \ldots, \xi_5$, detection of automorphism groups via the seven algebraic loci $L_1, \ldots, L_7$, systematic enumeration of rational functions of bounded height, derivation of explicit locus equations, and large-scale statistical and machine-learning analysis. Using this software we generated a dataset of 2 078 697 rational functions over $\mathbb{Q}$ with naive height at most 4, and demonstrated that a classifier based on the invariants achieves 99.992% accuracy.

All components are implemented in five interlinked SageMath/Python notebooks that share a common exact-arithmetic backend. Every result presented in this paper can be independently reproduced from these notebooks. The complete source code, together with the generated dataset, is publicly available on GitHub; see [2]

This work lays a foundation for systematic computational exploration of moduli spaces of rational maps. Future extensions of the framework to higher degrees are planned once the corresponding rings of invariants become fully known.

References

1. E. Badr, E. Shaska, and T. Shaska, *Rational Functions on the Projective Line from a Computational Viewpoint*, arXiv:2503.10835 [math.AG], 2026.
2. T. Shaska, *deg3-ratfuncs-moduli*: Software for Computing Moduli and Automorphism Groups of Degree-3 Rational Functions, GitHub repository, 2026. `https://github.com/sha-aims/deg3-ratfuncs-moduli`